

Universidade Tecnológica Federal do Paraná

Engenharia de Software - Curso de Arquitetura de Software (AS27S)

INSTRUTOR: Prof. Dr. Gustavo Santos

Aluno Juan Gabriel Benitez Bogo, RA 2601753

CCH - Design Patterns Builder

Link do projeto:

<https://github.com/juaNGb01/Trabalho-Arquitetura---Designs-Patterns.git>

Problema

Como criar uma aplicação que permita montar rotinas de exercícios de forma flexível, padronizada e fácil de manter?

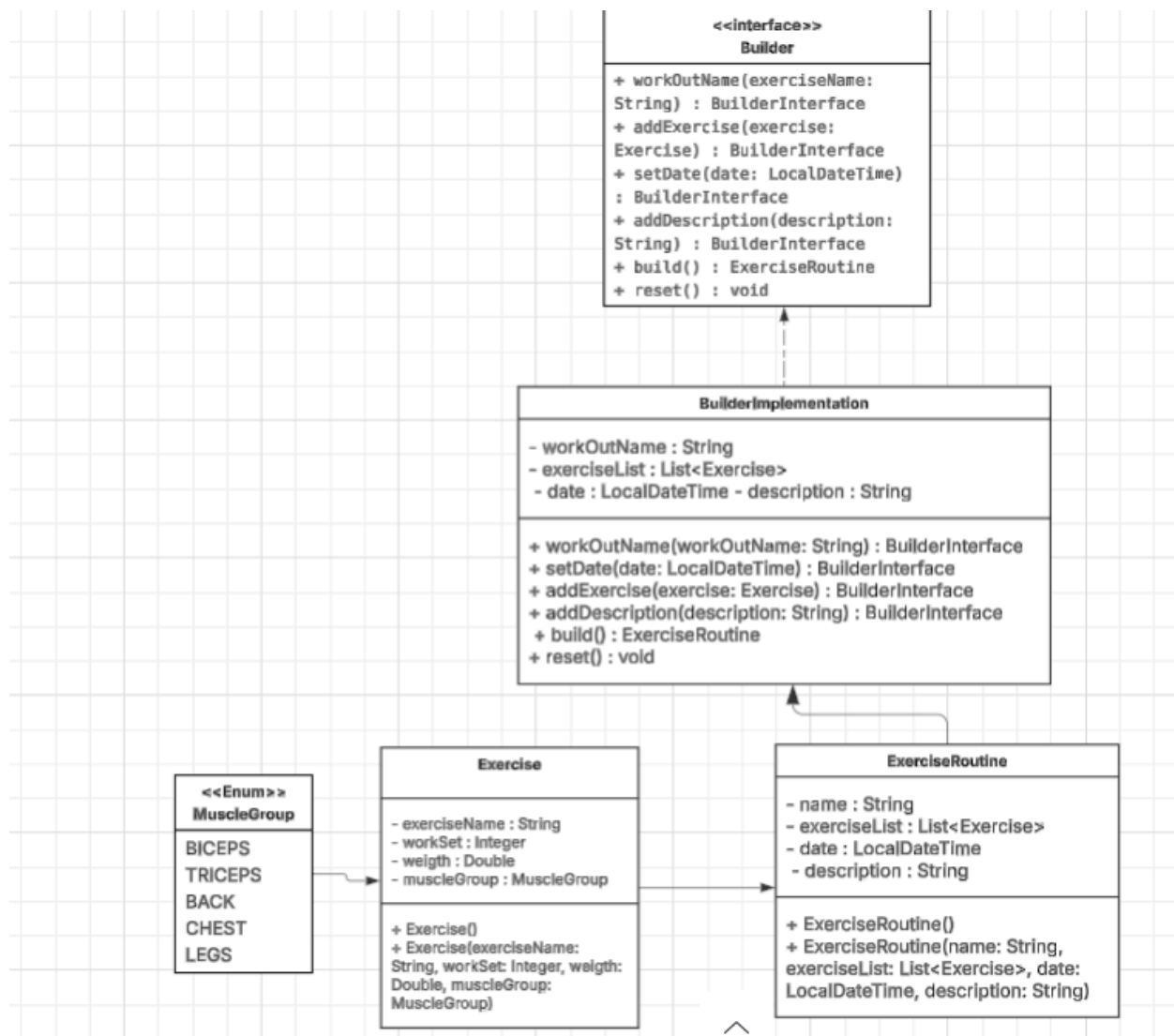
Descrição da Solução

A utilização desse padrão permitiu a criação de uma interface que define os métodos que serão utilizados durante a criação de um treino, como adicionar um novo exercício ao treino, ajustar nome do treino, data e descrição até o método build que irá gerar construir o treino criado.

Para implementar essa interface a classe BuilderImplementation() irá definir todos os atributos como que serão necessários para criar o treino de forma concreta(nome, data, descrição), além de uma lista que guarda todos os exercícios do treino em questão. Essa classe ao chamar o método build irá retornar uma nova instância da classe ExerciseRoutine, que representa o plano de treino criado

Além disso, foi criado também uma classe chamada Exercise que tem como único objetivo, definir atributos dos exercícios que compõem o treino, por exemplo, nome do exercício, peso, números de séries e grupo muscular do exercício.

Visão Geral [exemplo]



Digite seu texto aqui Digite seu texto aqui Digite seu texto aqui Digite seu texto aqui Digite seu texto aqui Digite seu texto aqui Digite seu texto aqui Digite seu texto aqui.

Exemplo de Código - Java

Interface Builder:

```
public interface BuilderInterface { 9 usages 1 implementation

    BuilderInterface workOutName(String exerciseName); 2 usages 1 implementation

    BuilderInterface addExercise(Exercise exercise); 4 usages 1 implementation

    BuilderInterface setDate(LocalDateTime date); 1 usage 1 implementation

    BuilderInterface addDescription(String description); 1 usage 1 implementation

    ExerciseRoutine build(); 2 usages 1 implementation
    void reset(); 1 usage 1 implementation
}
```

Implementação do builder

```
public class BuilderImplementation implements
BuilderInterface{

    private String workOutName;

    private List<Exercise> exerciseList = new ArrayList<>();

    private LocalDateTime date;

    private String description;
```

```
@Override

public BuilderInterface workoutName(String workoutName) {

    this.workoutName = workoutName;

    return this;

}

@Override

public BuilderInterface setDate(LocalDateTime date) {

    this.date = date;

    return this;

}

@Override

public BuilderInterface addExercise(Exercise exercise){

    exerciseList.add(exercise);

    return this;

}
```

```
    }

    @Override

    public BuilderInterface addDescription(String
description) {

        this.description = description;

        return this;

    }

    @Override

    public ExerciseRoutine build(){

        return new ExerciseRoutine(workOutName, exerciseList,
date, description);

    }

    @Override

    public void reset(){

        workOutName = null;

        exerciseList = new ArrayList<>();

    }

}
```

```
date = null;

description = null;
```

Classe exercise:

```
public class Exercise { 18 usages

    private String exerciseName; 4 usages
    private Integer workSet; 4 usages
    private Double weight; 4 usages
    private MuscleGroup muscleGroup; 4 usages

    public Exercise(){} 4 usages

    public Exercise(String exerciseName, Integer workSet, Double weight, MuscleGroup muscleGroup) { no usages
        this.exerciseName = exerciseName;
        this.workSet = workSet;
        this.weight = weight;
        this.muscleGroup = muscleGroup;
    }
}
```

```
public void setWeight(Double weight) { this.weight = weight; }

public MuscleGroup getMuscleGroup() { return muscleGroup; }

public void setMuscleGroup(MuscleGroup muscleGroup) { this.muscleGroup = muscleGroup; }

@Override
public String toString() {
    return String.format(
        "\n> %s" +
        "    \nSéries      : %d\n" +
        "Peso           : %.1f kg\n" +
        "Grupo Muscular: %s\n",
        exerciseName,
        workSet,
        weight,
        muscleGroup
    );
}
```

classe exeriseRoutine:

```
public class ExerciseRoutine { 7 usages
    private String name; 3 usages
    private List<Exercise> exerciselList = new ArrayList<>(); 3 usages
    private LocalDateTime date; 3 usages
    private String description; 3 usages

    public ExerciseRoutine(){} no usages

    public ExerciseRoutine(String name, List<Exercise> exerciselList, LocalDateTime date, String description) { 1 usage
        this.name = name;
        this.exerciselList = exerciselList;
        this.date = date;
        this.description = description;
    }

    //name
    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    //exercise list
    public List<Exercise> getExerciselList() { return exerciselList; }
```

```
//description
    public String getDescription() { return description; }

    public void setDescription(String description) { this.description = description; }

    @Override
    public String toString() {
        return "Plan Name: " + getName() +
            "\nDate: " + getDate() +
            "\nDescription: " + getDescription() +
            "\nList of exercise: " + getExerciselList();
    }
```

Main:

```
public class Main {

    public static void main(String[] args) {
```

```
//criação builder

BuilderImplementation builder = new BuilderImplementation();

//criação exercicios

Exercise ex = new Exercise();

ex.setExerciseName("Supino");

ex.setWorkSet(5);

ex.setWeigth(25.0);

ex.setMuscleGroup(MuscleGroup.CHEST);


Exercise ex2 = new Exercise();

ex2.setExerciseName("Triceps Frances");

ex2.setWorkSet(3);

ex2.setWeigth(20.0);

ex2.setMuscleGroup(MuscleGroup.TRICEPS);


//criação plano de treino 1

ExerciseRoutine exerciseRoutine = builder.workOutName("Triceps
and chest")

.addExercise(ex)
```

```
        .addExercise(ex2)

        .addDescription("New PR")

        .setDate(LocalDate.now())

        .build();

System.out.println(exerciseRoutine.toString());

builder.reset();

//criação exercicios 2

Exercise ex3 = new Exercise();

ex3.setExerciseName("Pulley frente");

ex3.setWorkSet(5);

ex3.setWeigth(25.0);

ex3.setMuscleGroup(MuscleGroup.BACK);

Exercise ex4 = new Exercise();

ex4.setExerciseName("remada curvada");

ex4.setWorkSet(3);
```

```
ex4.setWeigth(20.0);

ex4.setMuscleGroup(MuscleGroup.BACK);

//criação plano de treino 2

exerciseRoutine = builder.workOutName("Back Routine")

    .addExercise(ex3)

    .addExercise(ex4)

    .build();

System.out.println(exerciseRoutine.toString());

}

}
```

Consequências

- o padrão builder implementa um método reset, isso permite resetar a instância para utilizá-la em situações diferentes sem a necessidade de instanciar um novo builder.
- A criação de um treino mais simples ou mais complexo sempre segue a mesma estrutura
- Isolamento: as alterações para a criação de treinos é feita apenas no builder da aplicação, isso também permite uma maior organização do projeto, mantendo a lógica de construção centralizada